
Started on Thursday, 21 May 2026, 11:19 AM

State Finished

Completed on Thursday, 21 May 2026, 12:59 PM

Time taken 1 hour 40 mins

Grade **157.00** out of 400.00 (**39%**)

Question **1**

Partially correct

Mark 82.00 out of 100.00

Time limit	3 s
Memory limit	64 MB

Vending Machine

Setelah Kebin dan Stewart berhasil membantu pengelolaan kedai jus milik Dr. Neroifa. Dr. Neroifa ingin melakukan ekspansi usahanya dengan cara menjual jusnya di dalam Vending Machine yang terletak di daerah ramai penduduk. Anda diminta untuk membantu Dr. Neroifa agar vending machine jus dapat berjalan dengan baik pada: [VendingMachine.java](#).

Untuk memudahkan pengujian class, berikut program [Main](#) yang dapat Anda coba.

Berikut contoh input dan outputnya [File](#).

Format Pengumpulan

Kumpulkan file: [VendingMachine.java](#).

Pastikan program dapat dikompilasi dan dijalankan dengan benar.

Java 8

 [VendingMachine.java](#)

Score: 82

Blackbox

Score: 72

Verdict: Wrong answer

Evaluator: Exact

No	Score	Verdict	Description
1	9	Accepted	0.23 sec, 29.71 MB
2	9	Accepted	0.35 sec, 28.87 MB
3	9	Accepted	0.12 sec, 28.39 MB
4	0	Wrong answer	0.19 sec, 28.02 MB
5	0	Wrong answer	0.16 sec, 29.86 MB
6	9	Accepted	0.22 sec, 28.32 MB
7	9	Accepted	0.15 sec, 28.39 MB
8	9	Accepted	0.18 sec, 29.88 MB
9	9	Accepted	0.21 sec, 28.38 MB
10	9	Accepted	0.36 sec, 28.16 MB

Whitebox

Score: 10

No	Score	Checker	Description
1	10	PMD	

Question **2**

Not answered

Marked out of 100.00

Time limit	1 s
Memory limit	64 MB

Sliding Window Sum Paralel

Kebin sedang menganalisis deret angka besar. Ia ingin menghitung jumlah elemen setiap *sub-array* berurutan dengan panjang tetap (window). Karena datanya bisa sangat besar, Kebin membagi pekerjaan ke beberapa **thread**. Setiap thread menghitung sebagian window dan menyimpan hasilnya ke posisi yang sesuai.

Sliding window adalah teknik untuk mengambil potongan data berurutan dengan ukuran tetap, lalu menggeser potongan itu satu langkah demi satu langkah. Misal data = [1, 2, 3, 4, 5] dan ukuran window $W = 3$. Maka window yang terbentuk adalah:

- Window 0: [1, 2, 3] sum = 6
- Window 1: [2, 3, 4] sum = 9
- Window 2: [3, 4, 5] sum = 12

Jika panjang data N dan ukuran window W , maka jumlah window yang valid adalah $M = N - W + 1$.

Buatlah program yang menghitung *sum sliding window* secara paralel. Perbaikilah file [Main.java](#) dan [WindowThread.java](#)

Spesifikasi

- Jumlah setiap kemungkinan window adalah $M = N - W + 1$.
- Bagi semua window ke T threads dengan selisih ≤ 1 .
- Jika M tidak habis dibagi T , thread dengan indeks lebih kecil mendapat 1 window ekstra. Contoh: $M=8, T=3$, Banyak window per thread [3, 3, 2].
- Setiap thread menghitung sum setiap window dan menulis hasilnya ke array output.

Format Input

```
N
A1 A2 ... AN
W
T
```

Format Output

```
Thread 0:
window idx 0, sum = S0
window idx 1, sum = S1
...

Thread 1:
...

Thread T:
...
window idx (M-1), sum = S(M-1)
```

Contoh

Input

```
8
1 2 3 4 5 6 7 8
4
3
```

Output

```
Thread 0:  
window idx 0, sum = 10  
window idx 1, sum = 14
```

```
Thread 1:  
window idx 2, sum = 18  
window idx 3, sum = 22
```

```
Thread 2:  
window idx 4, sum = 26
```

Batasan

- $1 \leq N \leq 1.000.000$
- $1 \leq W \leq N$
- $1 \leq T \leq 64$
- Nilai elemen A dalam rentang int

Catatan

- Dilarang menggunakan busy-wait atau `Thread.sleep` untuk sinkronisasi.
- Kumpulkan Main.java dan WindowThread.java dalam satu zip file.

Java 8

Question **3**

Partially correct

Mark 75.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Plugin Loader**Latar Belakang**

Dalam pengembangan perangkat lunak yang modular, seringkali kita ingin menambahkan fitur baru tanpa harus mengubah kode utama. Hal ini biasanya dilakukan dengan sistem **Plugin**. Program utama akan mencari dan memuat kelas-kelas plugin secara dinamis saat dijalankan.

Pada tugas ini, Anda diminta untuk mengimplementasikan sebuah **PluginLoader** sederhana menggunakan **Java Reflection**.

File yang Disediakan

File	Deskripsi
Plugin.java	Interface yang harus diimplementasikan oleh setiap plugin. Memiliki method <code>void start()</code> .
PluginLoader.java	Kelas yang akan Anda lengkapi. Berisi method <code>loadPlugin</code> .
Main.java	Driver.

Tugas Anda

Lengkapi method berikut di dalam **PluginLoader.java**:

```
public static Plugin loadPlugin(String className) throws Exception
```

Spesifikasi:

1. Muat kelas berdasarkan `className` menggunakan Reflection.
2. Pastikan kelas tersebut merupakan implementasi dari interface **Plugin**.
3. Jika validasi pada poin 2 gagal, lempar `IllegalArgumentException` dengan pesan:
"Kelas <className> tidak mengimplementasikan interface Plugin"
4. Buat dan kembalikan *instance* dari kelas tersebut menggunakan *default constructor*.

Referensi Dokumentasi

- [Dokumentasi Kelas: java.lang.Class](#)
- [Dokumentasi Kelas: java.lang.reflect.Constructor](#)
- [Oracle Java Tutorial: The Reflection API](#)

Format Pengumpulan

Kumpulkan file **PluginLoader.java**

Contoh Input & Output

Misalkan terdapat kelas **ValidPlugin** yang mengimplementasikan **Plugin** dan **InvalidPlugin** yang tidak.

Input (className) Output / Exception

ValidPlugin	(Plugin berhasil dijalankan)
InvalidPlugin	<code>java.lang.IllegalArgumentException: Kelas InvalidPlugin tidak mengimplementasikan interface Plugin</code>
NonExistent	<code>java.lang.ClassNotFoundException: NonExistent</code>

Java 8

 [PluginLoader.java](#)

Score: 75

Blackbox

Score: 75

Verdict: Wrong answer

Evaluator: Exact

No	Score	Verdict	Description
1	25	Accepted	0.21 sec, 28.67 MB
2	25	Accepted	0.08 sec, 28.13 MB
3	0	Wrong answer	0.25 sec, 28.79 MB
4	25	Accepted	0.08 sec, 28.95 MB

Question **4**

Not answered

Marked out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: WeatherSystem.zip**[BONUS]****Latar Belakang**

Observer Pattern adalah salah satu *behavioral design pattern* yang mendefinisikan mekanisme *subscription*: satu objek (*subject/publisher*) menyimpan daftar objek lain (*observer/subscriber*) dan secara otomatis memberitahu mereka setiap kali terjadi perubahan pada state-nya. Pola ini memisahkan logika "apa yang berubah" dari logika "siapa yang perlu tahu", sehingga komponen menjadi longgar-kait (*loosely coupled*).

Struktur dasar Observer Pattern terdiri dari:

- **Subject (Publisher):** menyimpan daftar observer dan memanggil `update()` pada masing-masing observer setiap kali state berubah.
- **Observer (Subscriber):** mendefinisikan interface `update()` yang dipanggil oleh subject.
- **Concrete Observer:** implementasi nyata dari observer yang bereaksi terhadap notifikasi.

Deskripsi Soal

Kamu diminta membangun sistem pemantau cuaca berbasis **Observer Pattern**. Dalam sistem ini, sebuah *stasiun cuaca* (subject) menyimpan data suhu dan kelembaban. Setiap kali data cuaca diperbarui, semua tampilan yang terdaftar (*observer*) akan **otomatis diberitahu** dan menampilkan nilai terkini.

File `Main.java` (driver) sudah disediakan dan **tidak boleh diubah**. Tugas kamu adalah mengimplementasikan file-file [berikut](#):

- `WeatherObserver.java`: interface observer
- `WeatherStation.java`: subject yang mengelola daftar observer dan data cuaca
- `TemperatureDisplay.java`: observer yang menampilkan suhu
- `HumidityDisplay.java`: observer yang menampilkan kelembaban

Kumpulkan `WeatherSystem.zip` yang berisi semua file `.java` yang kamu buat, termasuk `Main.java`.

Spesifikasi**1. Interface `WeatherObserver`**

Interface yang mendefinisikan kontrak untuk semua observer cuaca.

Method	Keterangan
<code>void update(double temperature, double humidity)</code>	Dipanggil oleh subject saat data cuaca berubah. Implementasi masing-masing observer menentukan tampilannya.
<code>String getName()</code>	Mengembalikan nama unik observer ini.

2. Kelas `WeatherStation` (Subject / Observable)

Menyimpan data cuaca terkini dan mengelola daftar observer. Wajib memiliki field `observers` bertipe `List<WeatherObserver>` dan method `notifyObservers()` yang dipanggil setiap kali data berubah.

Method / Constructor	Keterangan
<code>WeatherStation()</code>	Inisialisasi stasiun dengan suhu=0.0, kelembaban=0.0, dan daftar observer kosong.
<code>void addObserver(WeatherObserver observer)</code>	Menambahkan observer ke daftar.
<code>boolean removeObserver(String name)</code>	Menghapus observer dengan nama yang sesuai. Mengembalikan <code>true</code> jika berhasil, <code>false</code> jika tidak ditemukan.
<code>boolean hasObserver(String name)</code>	Mengembalikan <code>true</code> jika observer dengan nama tersebut sudah terdaftar.
<code>void setMeasurements(double temperature, double humidity)</code>	Memperbarui data cuaca lalu memanggil <code>notifyObservers()</code> .
<code>private void notifyObservers()</code>	Memanggil <code>update(temperature, humidity)</code> pada setiap observer dalam urutan pendaftaran.
<code>double getTemperature()</code>	Mengembalikan suhu saat ini.

Method / Constructor	Keterangan
<code>double getHumidity()</code>	Mengembalikan kelembaban saat ini.
<code>int getObserverCount()</code>	Mengembalikan jumlah observer yang terdaftar.

3. Kelas `TemperatureDisplay` (Concrete Observer)

Observer yang mencetak suhu saat dipanggil oleh stasiun cuaca.

Method / Constructor	Keterangan
<code>TemperatureDisplay(String name)</code>	Membuat tampilan suhu dengan nama yang diberikan.
<code>void update(double temperature, double humidity)</code>	Mencetak: <code>Display <nama>: Suhu <suhu></code> Format suhu: satu angka desimal (<code>%.1f</code>). Contoh: <code>Display TempA: Suhu 25.0</code>
<code>String getName()</code>	Mengembalikan nama tampilan ini.

4. Kelas `HumidityDisplay` (Concrete Observer)

Observer yang mencetak kelembaban saat dipanggil oleh stasiun cuaca.

Method / Constructor	Keterangan
<code>HumidityDisplay(String name)</code>	Membuat tampilan kelembaban dengan nama yang diberikan.
<code>void update(double temperature, double humidity)</code>	Mencetak: <code>Display <nama>: Kelembaban <kelembaban>%</code> Format kelembaban: satu angka desimal (<code>%.1f</code>). Contoh: <code>Display HumidA: Kelembaban 60.0%</code>
<code>String getName()</code>	Mengembalikan nama tampilan ini.

Constraint Whitebox

Implementasi kamu akan divalidasi secara struktural oleh PMD. Pastikan:

- `WeatherStation` memiliki field bernama `observers`.
- `WeatherStation` memiliki method bernama `notifyObservers`.
- Method `setMeasurements` di `WeatherStation` memanggil `notifyObservers()`.

Format Masukan

Baris pertama berisi integer `Q` (jumlah perintah). Diikuti `Q` baris perintah:

- `REGISTER <TEMP_DISPLAY|HUMIDITY_DISPLAY> <nama>`: daftarkan observer baru
- `REMOVE <nama>`: hapus observer dari daftar
- `UPDATE <suhu> <kelembaban>`: perbarui data cuaca (memicu notifikasi ke semua observer)
- `STATUS`: tampilkan informasi stasiun

Format Keluaran

Perintah	Kondisi	Keluaran
REGISTER	berhasil	OK
REGISTER	nama sudah terdaftar	ALREADY_REGISTERED
REGISTER	tipe tidak dikenal	UNKNOWN_OBSERVER_TYPE
REMOVE	berhasil	OK
REMOVE	nama tidak ditemukan	NOT_FOUND
UPDATE	-	Setiap observer mencetak satu baris (dalam urutan pendaftaran). <code>TemperatureDisplay: Display <nama>: Suhu <suhu></code> <code>HumidityDisplay: Display <nama>: Kelembaban <kelembaban>%</code> Jika tidak ada observer, tidak ada keluaran.
STATUS	-	<code>Station: Suhu=<suhu>, Kelembaban=<kelembaban>%, Observer=<jumlah></code>
Perintah tidak dikenal	-	UNKNOWN_COMMAND

Semua nilai suhu dan kelembaban dicetak dengan satu angka desimal (contoh: `25.0`, `60.0`).

Contoh Masukan dan Keluaran

Masukan 1:

```
5
REGISTER TEMP_DISPLAY TempA
REGISTER HUMIDITY_DISPLAY HumidA
STATUS
UPDATE 25.0 60.0
STATUS
```

Keluaran 1:

```
OK
OK
Station: Suhu=0.0, Kelembaban=0.0%, Observer=2
Display TempA: Suhu 25.0
Display HumidA: Kelembaban 60.0%
Station: Suhu=25.0, Kelembaban=60.0%, Observer=2
```

Masukan 2:

```
8
REGISTER TEMP_DISPLAY TA
REGISTER HUMIDITY_DISPLAY HA
UPDATE 20.0 50.0
REMOVE TA
STATUS
UPDATE 25.0 65.0
REMOVE HA
STATUS
```

Keluaran 2:

```
OK
OK
Display TA: Suhu 20.0
Display HA: Kelembaban 50.0%
OK
Station: Suhu=20.0, Kelembaban=50.0%, Observer=1
Display HA: Kelembaban 65.0%
OK
Station: Suhu=25.0, Kelembaban=65.0%, Observer=0
```

Masukan 3:

```
6
REGISTER TEMP_DISPLAY Screen1
REGISTER TEMP_DISPLAY Screen1
REGISTER HUMIDITY_DISPLAY Screen1
STATUS
UPDATE 25.0 60.0
STATUS
```

Keluaran 3:

```
OK
ALREADY_REGISTERED
ALREADY_REGISTERED
Station: Suhu=0.0, Kelembaban=0.0%, Observer=1
Display Screen1: Suhu 25.0
Station: Suhu=25.0, Kelembaban=60.0%, Observer=1
```

Java 8

[◀ Slide Tutorial 6](#)

Jump to...